Parallel Programming Semester Recap

A detailed summary of all concepts discussed in the Parallel Programming lecture in the Basisjahr's second semester at ETH Zürich.

Erik Girdauskas

June 1, 2026

Table of Contents

Processes and Java Threads 1
The keyword synchronized 5
Parallel Architecture 8
Pipelining 10
Amdahl & Gustafson 12
Divide and Conquer, ExecutorService 14
ForkJoin Parallelism, Task Graphs 16
Parallel Algorithms and Sorting 20
Locks, Synchronization, and Race Conditions 23
Exam Tasks for the Rest of Part 1 26
Java Memory Model 30
Implementing Mutual Exclusion 31
Atomic Operations and Deadlocks 34
Semaphores and Barriers 35
Producer-Consumer Routine 37
Monitors 37
Reader-Writer Locks 39
Lock Granularity 40
ABA Problem 46
Linearizibility and Sequential Consistency 47
Consensus 50
Transactional Memory 51
Distributed Memory and MPI 55
Exam Tasks for Part 2 58

color code: topic, theorem, important stuff

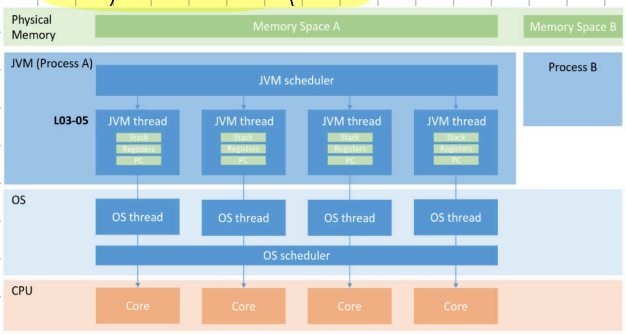
PProg Semester Recap

by Erik Gintautas

Key formula: speedup (using p processors) = $\frac{\text{execution time (using 1 processor)}}{\text{execution time (using p processors)}}$

Java Threads:

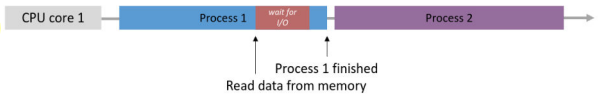
Image of all components:



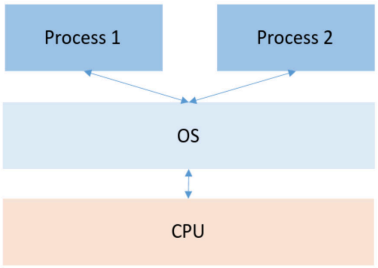
→ A process is a program executing in an OS (simplified)

→ each running instance (example: each browser window) is its own process

On a single-core CPU:



→ sequential execution, one process executes at a time because our example CPU can only execute one instruction at a time



→ if a process is waiting on I/O, the CPU is idle until the process can continue executing

→ in reality, we have multitasking even on a single-core CPU:

→ the OS scheduler handles task management, the CPU scheduler decides which process gets CPU time

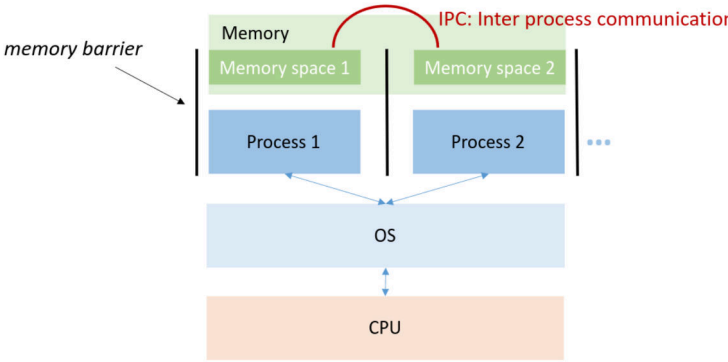
→ this allows asynchronous I/O → we have I/O and CPU executing in parallel, overall progress is not bottlenecked by I/O at all times

→ Multitasking allows for concurrency: multiple processes are active at the same time and make progress, not necessarily simultaneously

→ notice how the image says context switch? but what is a context?

→ processes share a CPU but have their own memory space

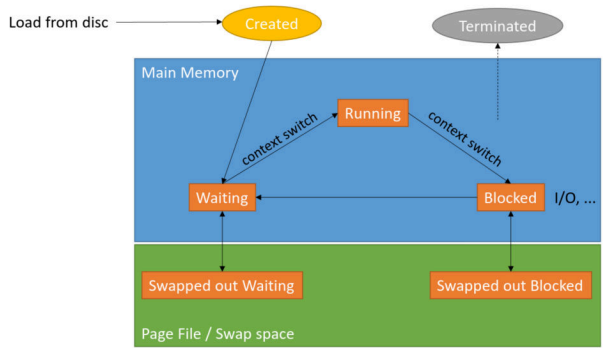
→ a process's context splits up into the HW context (instruction counter, CPU registers, execution state ...), memory context (virtual memory, stack, heap, ...) and OS-level context (Process ID, open files, ...)



→ switching contexts causes a large overhead when switching processes → contexts are stored in main memory, OS can manage relatively efficiently

→ PCBs are OS-level and offer information on all of these contexts

Process lifecycle states



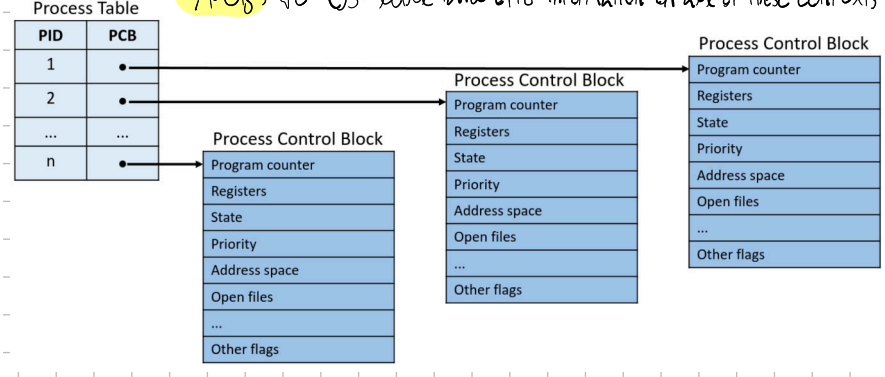
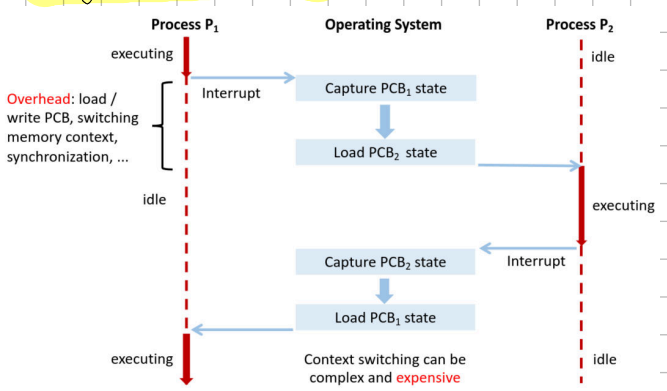
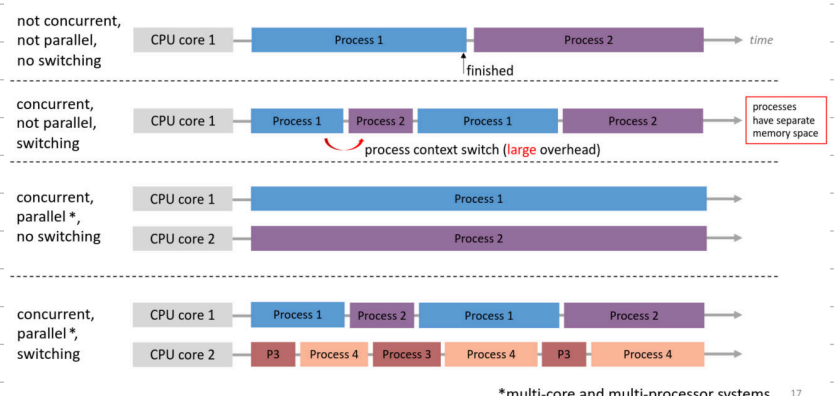


Image for context switch:

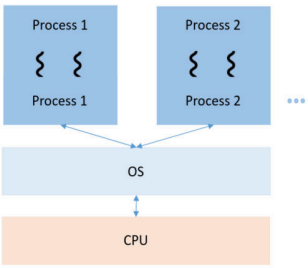


Def. parallelism: multiple processes execute simultaneously on different CPU cores



Multithreading:

→ a process can have multiple user-level threads that act as units of computation in the process and are managed by a user-level library

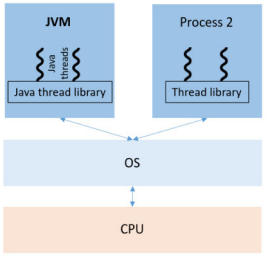


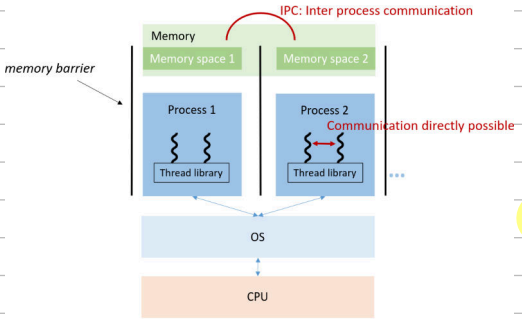
→ multithreading takes full advantage of multi-core (or CPU) system, for example it's useful in reactive systems like GUI apps with user input

Threads: Independently executing sequences running in same OS process

→ multiple threads share address space (memory),

→ this means they are not protected from each other and can communicate easier





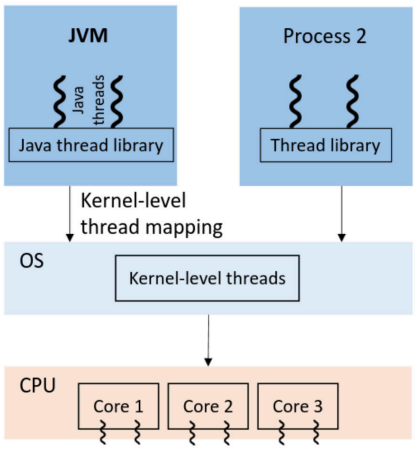
→ Context switching between threads is efficient because we are in the same address space, no OS-level PCB storing/loading or scheduling is needed

Thread levels:

User-level: managed by app using a thread library → JVM does this in Java, Kernel doesn't see the threads

Kernel-level: managed by OS, in modern JVM, Java threads map one-to-one to OS threads, they can be scheduled on different cores

CPU-level: execute threads simultaneously on physical cores (simultaneous multithreading, hyperthreading)



Java Threads:

→ Every Java program runs inside a JVM process and has at least one execution thread: the main thread

→ A JVM process can contain multiple created threads, which execute concurrently

→ Threads share their memory space (heap), but each thread has its own execution stack containing method calls/local variables and its own instruction stream (independent exec. units within a process)

Creating Java Threads:

Archaic option: instantiate a subclass of java.lang.Thread

```
class ConcurrWriter extends Thread { …
    public void run() {
        // if multiple threads started -> concurrent execution
    }
}
ConcurrWriter writerThread = new ConcurrWriter();
writerThread.start();    // calls ConcurrWriter.run()
```

→ you need to override the run() method

→ run() is called when the thread begins to execute

start() is needed to start the thread → it invokes run()

```
class ConcurrWriter extends Thread { …

    public void run() {
        // if multiple threads started -> concurrent execution
        System.out.println("Hello world");
    }
}

ConcurrWriter writerThread1 = new ConcurrWriter();
ConcurrWriter writerThread2 = new ConcurrWriter();

writerThread1.start();    // calls ConcurrWriter.run()
writerThread2.start();    // calls ConcurrWriter.run()
```

writerThread1 and writerThread2 executes run() independently

objects live on heap

both threads we created have own stacks

→ a thread terminates when run() returns

→ calling run() does not create a thread

→ a start call causes the JVM to: create a call stack for the thread, assign a program counter, schedule it for execution, and run run() on its own stack

Better option: Implement java.lang.Runnable

```
public class ConcurrWriter implements Runnable {
    …
    public void run() { …
     // if multiple threads started -> concurrent execution
     }
}

ConcurrWriter writerTask = new ConcurrWriter(); // task
Thread t = new Thread(writerTask);              // executor
t.start();    // calls ConcurrWriter.run()
```

Clear separation between executor and task

→ one method: run()

```
class ConcurrWriter implements Runnable {
    int x = 0;

    public void run() {
        // if multiple threads started -> concurrent execution
        System.out.println("Hello world");
    }
}

ConcurrWriter writerTask = new ConcurrWriter();

Thread t1 = new Thread(writerTask);
Thread t2 = new Thread(writerTask);

t1.start();    // calls ConcurrWriter.run()
t2.start();    // calls ConcurrWriter.run()
```

Shared instance variable (both threads use the same task object)

One task object (heap)

Two thread objects (heap)

→ each start() call creates an execution thread

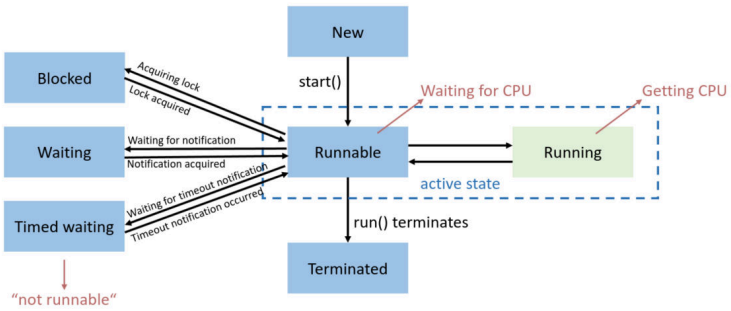
→ program ends when all non-daemon threads finish

→ threads can keep running after main() returns

→ creating a Thread object or calling run() doesn't start a thread in itself

→ this image makes much more sense later on in the semester

Life cycle of a Thread



Interleaving:

→ Threads can execute at the same physical time on diff. cores

→ no matter how parallel the execution, individual operations will complete at different moments in time
⇒ operations of multiple threads form a single global timeline

→ operations of multiple threads can be interleaved in many ways

→ concurrent programs can have multiple execution orders
⇒ exec. order is non-deterministic

Def. Interleaving: Given multiple threads executing a sequence of instructions, an interleaving is a sequence of instructions obtained by merging the individual instruction sequences. The order of instructions within each thread is preserved, but instructions from different threads can appear in any order.

Def. Non-deterministic order: Refers to the sequence of operations being performed by different threads not being fixed or predictable. The execution order can vary because of things like thread scheduling, system load, or OS-level thread management. We will find out when this is problematic.

Thread Attributes:

Thread ID: unique ID, accessible with t.getId() (assume T is instance of Thread)

Name: get thread name with t.getName(), set with t.setName()

Priority: can be between 1 and 10, get with t.getPriority(), set with t.setPriority() (use constants like Thread.MAX_PRIORITY)

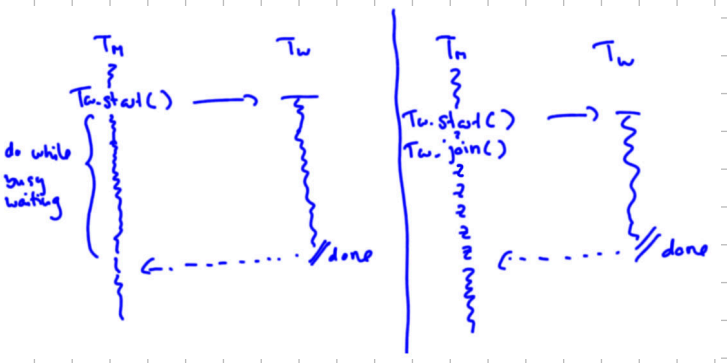
Status: can be new, runnable, blocked, waiting, time waiting, terminated, example: if (t.getState() == State.TERMINATED)

→ High-priority threads usually, but not always, finish before low-priority ones because the OS scheduler, not the JVM, decides which thread gets CPU time

Joining Threads:

→ often, a main thread starts many workers and needs their results

→ without join(), it would need to spin until each worker is TERMINATED → inefficient, uses CPU time




```
...
finish = false;
while (!finish) {
  ...
  finish = true;
  for (int i=0; i<10; i++){
    finish = finish && (threads[i].getState() == State.TERMINATED);
  }
}
```

→ Joining: send the main thread to sleep and wake it up once results are ready

```
...
for (int i=0; i<10; i++) {
  threads[i].join(); // May throw InterruptedException
}
```

→ joining brings about context switch overhead and if worker threads are short-lived, busy waiting may perform better

→ The main thread may not know about exceptions in worker threads, join() is unaffected by them

```
public class Worker extends Thread {
  Data result;
  ...

  @Override
  public void run() {
    ...
    // someObject could be null → NPE
    result = calculate(someObject.getData());
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    Worker worker = new Worker(...);
    worker.start();

    worker.join(); // Unaffected
    println(worker.result); // Another NPE
  }
}
```

you can handle this with: registering an exception handler with Thread or ThreadGroup, or using setDefaultUncaughtExceptionHandler() for all threads:

```
public class ExceptionHandler
    implements UncaughtExceptionHandler {

  public Set<Thread> threads = new HashSet<>();

  @Override
  public void uncaughtException(Thread thread,
                                Throwable throwable) {

    println("An exception has been captured");
    println(thread.getName());
    println(throwable.getMessage());
    ...
    threads.add(thread);
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    ...

    ExceptionHandler handler = new ExceptionHandler();

    thread.setUncaughtExceptionHandler(handler);

    ...

    thread.join();

    if (handler.threads.contains(thread)) {
      // bad
    } else {
      // good
    }
  }
}
```

Code Example: Interrupt

```
public static void main(String[] args) {
        Thread buggyThread = new Thread(new RunnableThatTakesForever());
        buggyThread.start();
        …
        buggyThread.join();
        …
}
```

thread sleeps forever – main waits forever

```
private static class RunnableThatTakesForever implements Runnable {
    public void run() {
        try {
                // This is a bug: Threads takes forever to finish
                Thread.sleep(10000000000L);
            } catch (InterruptedException e) {
          …
            }
    }
}
```

```
private static class RunnableThatTakesForever implements Runnable {
   public void run() {
        try {
                Thread.sleep(10000000000L); // Bug
        } catch (InterruptedException e) {
                // interrupt exception in buggyThread
                System.out.println("Interrupted exception RunnableThatTakesForever");
        }
   }
}
```

For the interrupt mechanism to work correctly, the interrupted thread must support its own interruption

```
public static void main(String[] args) {
        Timer timer = new Timer();
        Thread buggyThread = new Thread(new RunnableThatTakesForever());
        buggyThread.start();
        timer.schedule(new MyTimerTask(buggyThread), 8000);

        …
        buggyThread.join();
        …
}
```

calls interrupt on buggyThread

```
private static class MyTimerTask extends TimerTask {
        …
        public void run() {
                System.out.println("Now interrupting the other thread");
                thread.interrupt();
        }
}
```

invoking interrupt on the thread object

Shared Resources: (example)

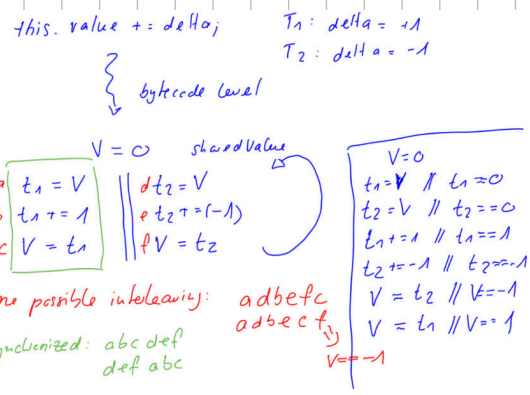
Synchronized incrementing and decrementing

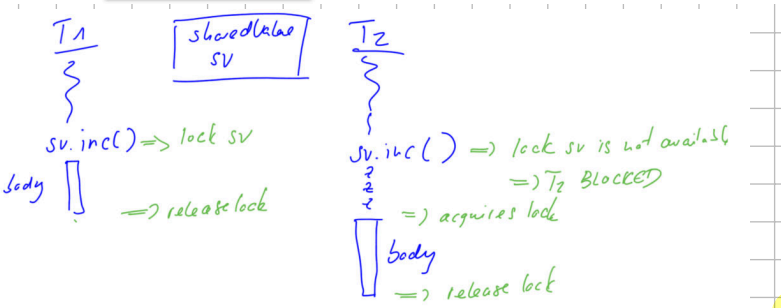
```
public class Counter implements Runnable {
  public int ticks = -1;

  private Cell cell;
  private int delta;
  private int maxTicks;

  Counter(Cell cell, int delta, int maxTicks) {
    this.cell = cell;
    this.delta = delta;
    this.maxTicks = maxTicks;
  }

  @Override
  public void run() {
    ticks = 0;

    while (ticks < maxTicks) {
      cell.inc(delta);
      ++ticks;
    }
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    ...

    Counter up = new Counter(cell, 1, MAX_TICKS);
    Counter down = new Counter(cell, -1, MAX_TICKS);

    Thread upWorker = new Thread(up);
    Thread downWorker = new Thread(down);

    upWorker.start(); downWorker.start();
    upWorker.join();  downWorker.join();

    System.out.printf("Cell value:  %d\n", cell.get());
  }
}
```

```
public class Cell {
  private long value;

  ...

  public void inc(long delta) {
    this.value += delta;
  }
}
```

```
Cell value: -799
Cell value: 667088
Cell value: -281765
Cell value: 147854
...
```





75

Updating shared state in parallel:

```
// relevant bytecode
ALOAD 0
DUP
GETFIELD LongCell.value
LLOAD 1
LADD
PUTFIELD LongCell.value
```

→ the statement this.value += delta takes all these steps

→ many different interleavings

→ data race (low-level race condition): unsynchronized read/write access to value, which is shared → here, inc() is the critical section

Definitions: Data race: erroneous program behavior caused by insufficiently synchronized access to a shared resource by multiple threads (simultaneous read/write or write/write of same mem. location)

Bad interleaving (high-level race condition): Erroneous program behavior caused by unfavorable interleaving

Critical section (CS): part of program where shared resources are accessed by multiple threads, only one thread should execute it at a time to prevent any race conditions

Thread safety: implies program safety, typically: there are no bad interleavings, hard to achieve, and needs parallel design

Liveness: "there is eventual progress", example: endless loops are liveness hazards

Performance hazards: things like frequent context switches, loss of locality, mutual exclusion through locking

The keyword synchronized:

→ we avoid bad interleaving with explicit synchronization

→ all objects have an intrinsic (monitor) lock → synchronized operations lock the object → when it is locked, no other thread can lock it (this is not a semaphore)

→ the conservative, safe way of ensuring thread safety when multiple threads access shared memory and at least one writes is to lock the object in shared memory

→ the sync. block enforces mutual exclusion on some object

→ intrinsic lock: any Java object can be a lock

```
// synchronized block: uses the given object as a lock
synchronized (object) {
    statement(s); // critical sections
}
```

→ some thread T_1 asks to run a block of code synchronized to some object O

→ if no thread locked O, T_1 locks O and executes the CS

→ if some thread T_2 locked O, T_1 becomes blocked and must wait until T_1 is finished with O and unlocks it

→ then, T_1 is woken up and can proceed

```
// synchronized method: locks on "this" object
public synchronized type name(parameters) { ... }

// synchronized static method: locks on the given class
public static synchronized type name(parameters) { ... }
```

→ use synchronized methods like this when entire bodies are CS

→ If you go back to the code example from earlier, you will see if works now because of synchronization

A synchronized method, e.g.

```
public synchronized void inc(long delta) {
  this.value += delta;
}
```

is syntactic sugar for

```
public void inc(long delta) {
  synchronized (this) {
    this.value += delta;
  }
}
```

Synchronized incrementing and decrementing

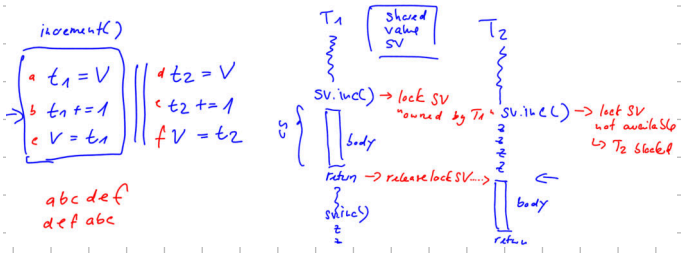
```
public class Counter implements Runnable {
  public int ticks = -1;

  private Cell cell;
  private int delta;
  private int maxTicks;

  Counter(Cell cell, int delta, int maxTicks) {
    this.cell = cell;
    this.delta = delta;
    this.maxTicks = maxTicks;
  }

  @Override
  public void run() {
    ticks = 0;

    while (ticks < maxTicks) {
      cell.inc(delta);
      ++ticks;
    }
  }
}
```

```
public class Main {
  public static void main(String[] args) {
    ...

    Counter up = new Counter(cell, 1, MAX_TICKS);
    Counter down = new Counter(cell, -1, MAX_TICKS);

    Thread upWorker = new Thread(up);
    Thread downWorker = new Thread(down);

    upWorker.start(); downWorker.start();
    upWorker.join();  downWorker.join();

    System.out.printf("Cell value:   %d\n", cell.get());
  }
}
```

```
public class Cell {
  private long value;

  ...

  public synchronized void inc(long delta) {
    this.value += delta;
  }
}
```

```
Cell value: 0
Cell value: 0
Cell value: 0
Cell value: 0
...
```

Note: Since all thread work involves calling inc(), defining it as critical section reduces parallism



Def.: Mutual exclusion: Only one process or thread enters the CS at a time, which prevents either type of race condition

Atomicity: A critical section is executed as an indivisible unit (here, protected by synchronized), preventing other threads from interrupting or seeing partial updates

Reentrance of locks:

A thread can request to lock an object it has already locked

```
public class Foo {
    public void synchronized f() { … }
    public void synchronized g() { … f(); … }
}

Foo foo = new Foo();

synchronized(foo) { … synchronized(foo) { … } … }
```

share the same lock if called on the same instance

Granularity of synchronization:

→ fine-grained locks are more performant but you have to watch out for correctness

```
public class SynchronizedCounter {
  private int c = 0;
  public synchronized void increment() { c++; }
  public synchronized void decrement() { c--; }
  public synchronized int value() { return c; }
}
```

```
public void addName(String name) {  synchronized(this) {
    lastName = name;
    nameCount++;
  }
  nameList.add(name); // add synchronizes on nameList
}
```

Interleaving example:

→ 2 is our desired result:

```
                c = 0

T1:                          T2:

synchronized (this)          synchronized (this)
{                            {
 1:  t1 = c;                  4: t2 = c;
 2:  t1++                     5: t2++
 3:  c = t1;                  6: c = t2;
}                            }
```

Possible interleavings: 123456 or 456123

```
                c = 0

T1:                          T2:

synchronized (this)
{
 1:6 t1 = c;                  4: t2 = c;
 2:  t1++                     5: t2++
 3:  c = t1;                  6: c = t2;
}
```

→ unwanted interleaving causes c=1: 412356

```
            c = 0

T1:                          T2:

synchronized (this)          synchronized (p)
{                            {
 1:  t1 = c;                   4: t2 = c;
 2:  t1++                      5: t2++
 3:  c = t1;                   6: c = t2;
}                            }
```

→ this still allows the bad interleaving because T2 doesn't synchronize this

→ what if there's an exception in the sync block?

→ First, the synchronized on the given object is released

→ then, the exception is caught and the exception handler is executed → if there is no handler, the exception is propagated to the caller → code after the exception is never executed and computations before it are never reversed

Producer-Consumer: both run indefinitely, producer puts items into shared buffer, consumer removes them

V1:

Producer-Consumer: v1

```
public class UnboundedBuffer {
  // Internal implementation could be a standard collection,
  // or a manually-maintained array or linked-list

  public boolean isEmpty() { ... }
  public void add(long value) { ... }
  public long remove() { ... }
}
```

```
public static void main(String[] args) {
  UnboundedBuffer buffer = new UnboundedBuffer();

  Producer producer = new Producer(buffer);
  producer.start();

  Consumer[] consumers = new Consumer[3];

  for (int i = 0; i < consumers.length; ++i) {
    consumers[i] = new Consumer(i, buffer);
    consumers[i].start();
  }
}
```

```
public class Consumer extends Thread {
  private final UnboundedBuffer buffer;
  ...

  public void run() {
    while (true) {
      while (buffer.isEmpty()); // Spin until item available
      performLongRunningComputation(buffer.remove());
    }
  }
}
```

```
public class Producer extends Thread {
  private final UnboundedBuffer buffer;
  ...

  public void run() {
    ...

    while (true) {
      prime = computeNextPrime(prime);
      buffer.add(prime);
    }
  }
}
```

Can you see any problems?

Problem: buffer could be empty between isEmpty() and remove()

Problem: buffer operations (add, remove) may be interleaved on bytecode level → internal state can corrupt

V2:

```
public class Consumer extends Thread {
  ...

  public void run() {
    long prime;
    while (true) {
      synchronized (buffer) {
        while (buffer.isEmpty());
        prime = buffer.remove();
      }
      performLongRunningComputation(prime);
    }
  }
}
```

```
public class Producer extends Thread {
  ...

  public void run() {
    ...

    while (true) {
      prime = computeNextPrime(prime);
      synchronized (buffer) {
        buffer.add(prime);
      }
    }
  }
}
```

→ synchronize for mutex in critical sections

Problem:
1. Consumer locks buffer
2. Consumer spins on isEmpty()
3. Producer waits for lock to free up
4. Deadlock! P and C wait for each other

V3:

```
public class Consumer extends Thread {
  ...

  public void run() {
    long prime;
    while (true) {
      synchronized (buffer) {
        while (buffer.isEmpty())
          buffer.wait();
        prime = buffer.remove();
      }
      performLongRunningComputation(prime);
    }
  }
}
```

```
public class Producer extends Thread {
  ...

  public void run() {
    ...

    while (true) {
      prime = computeNextPrime(prime);
      synchronized (buffer) {
        buffer.add(prime);
        buffer.notifyAll();
      }
    }
  }
}
```

buffer.wait(): Consumer goes to sleep (Status NOT RUNNABLE) and gives up buffer lock

buffer.notifyAll(): All threads waiting for buffer lock are woken up (status RUNNABLE)

wait() methods: releases object lock, thread waits on internal queue

notify(): wakes up an arbitrary thread waiting on object monitor

notifyAll(): wakes up all waiting threads → Threads compete non-deterministically, may not be fair (low-priority threads may never get access)

→ may only be called on locked objects

Why do we need synchronized and a loop?

Case 1: Assume exec. order 1, 2, 3: Now assume (bytecode) low-level bad interleavings are not a problem. What if the consumer checks and consumable() returns false?

```
public void consume() {
  1) if (!consumable()) {          (no loop, no synchronized)
  3)     wait();
     }   // release lock and wait for resource
     ... // have exclusive access to resource, can consume
}

public void produce() {
    ... // do something to make consumable() return true
  2) notifyAll();     // tell waiting threads to try consuming
     //  can also call notify() to notify one thread at a time
}
```

→ before calling wait(), produce completes, consume goes to wait()

→ if produce doesn't run again, consume is blocked forever even though there is something to consume

→ Java would throw an exception if any of these is called without synchronize

→ Case 2:

```
public synchronized void consume() {
    if (!consumable()) {
        wait();
    }   // release lock and wait for resource
    …   // have exclusive access to resource, can consume
}

public synchronized void produce() {
    … // do something to make consumable() return true
    notifyAll();    // tell waiting threads to try consuming
    //  can also call notify() to notify one thread at a time
}
```

→ Problem: Consumer can return from wait() for reasons other than being notified (example thread interrupt, this is a spurious wake-up)

→ we need to recheck the consumable condition after returning from wait, else we don't know why it returned → while loop strongly recommended

Nested lockout problem:

```
class Stack {
    LinkedList list = new LinkedList();
    public synchronized void push(Object x) {
      synchronized(list) {
        list.addLast( x ); notify();
    } }
    public synchronized Object pop() {
      synchronized(list) {
        if( list.size() <= 0 ) wait();
        return list.removeLast();
    } }
}
```

releases lock on this but not on list, a push from another thread will deadlock

Preventing it: no blocking code in synchronized methods, or provide non-sync. method of blocking object

Parallel Architecture: (I'll keep it short as these are overlaps with DDCA)

Programs compile to assembly instructions:

A program is just a list of instructions

```
public class Main {
    public static void main(String[] args) {
        float a;
        float x = 2;
        float y = 4;
        float z = 6;

        a = x * x + y * y + z * z;
    }
}
```

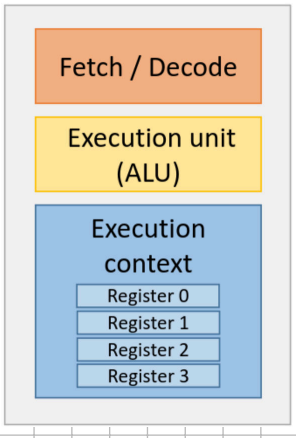
compile

JVM's instruction pointer

JVM's instruction stream

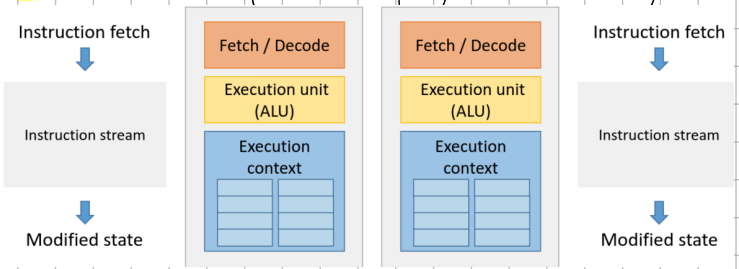
```
Java bytecode:
0:  iconst_2        // Push 2 onto the stack
1:  f2d             // Convert to float
2:  fstore_1        // Store float in local variable 1 (x)
3:  iconst_4        // Push 4 onto the stack
4:  f2d             // Convert to float
5:  fstore_2        // Store float in local variable 2 (y)
6:  bipush 6        // Push 6 onto the stack
7:  f2d             // Convert to float
8:  fstore_3        // Store float in local variable 3 (z)
9:  fload_1         // Load x from local variable 1
10: fload_1         // Load x again
11: fmul            // x * x
12: fload_2         // Load y from local variable 2
13: fload_2         // Load y again
14: fmul            // y * y
15: fadd            // (x * x) + (y * y)
16: fload_3         // Load z from local variable 3
17: fload_3         // Load z again
18: fmul            // z * z
19: fadd            // ((x * x) + (y * y)) + (z * z)
20: fstore_0        // Store the result in a (local variable 0)
21: return          // Return from main
```

→ as you know from DDCA, the basic structure of a processor is: fetch/decode ins., use ALU's or other functional units to perform the operation specified by the ins. while maintaining program state in registers



→ Instruction throughput is improved by exploiting ILP (ins. level parallelism)

→ this is done with superscalar execution, out-of-order execution, pipelining, and SIMD (all explained on DDCA sheet in detail) → this and Moore's law caused exponentially more efficient computation of programs in the single-core era



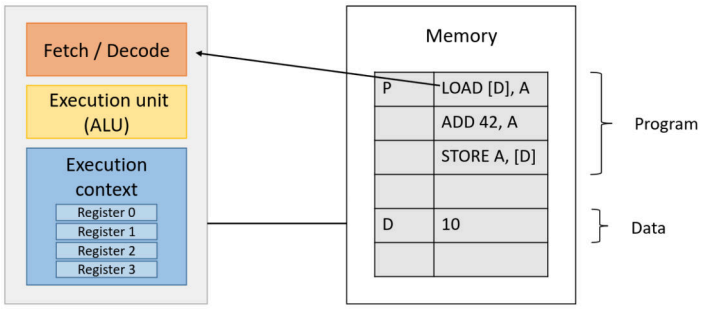
→ around the early 2000's, we were beginning to approach physical limits for things like transistor size

→ in multi-core systems, each core fetches and executes independently

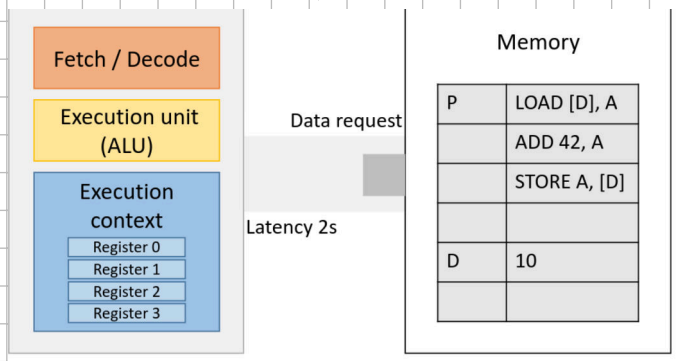
→ still, code compiles to an instruction stream designed for one core → solution: parallel programming into independent threads to maximize performance

Caching:

→ in the von Neumann model, program data and instructions use the same memory

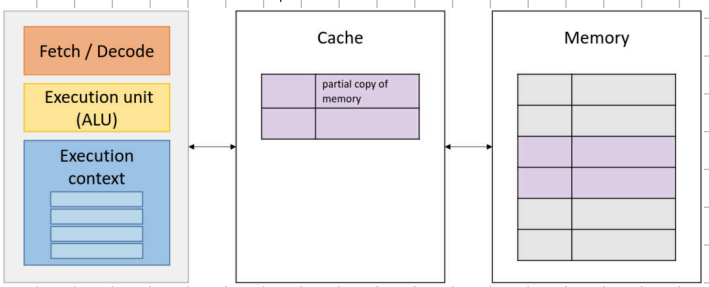


→ Mem. access latency: # of clock cycles it takes the system to provide data to processor



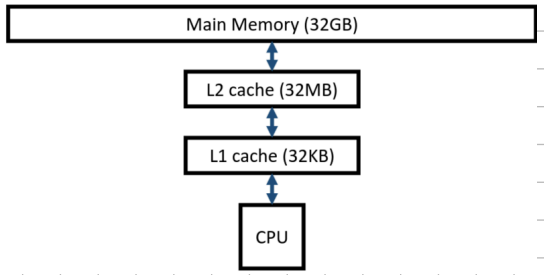
→ With larger CPUs and memories, processor stalls (not being able to run the next instruction) become more common, a major reason is accessing memory → solution: caches are smaller and faster

→ They reduce mem. access latency by exploiting data locality (spatial locality by storing locations close to what was loaded up, temporal by storing recently accessed locations)



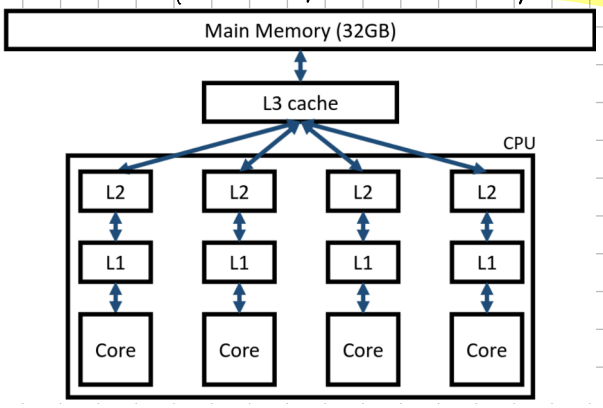
Memory hierarchy:

→ the faster SRAM used in caches is more expensive, so systems have less cache than main mem.



→ between cache levels, smaller caches allow for faster lookups

→ this is complicated by multi-core systems:



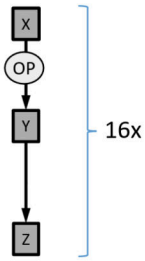
→ each core has its own caches

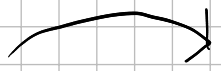
Compute Optimization:

→ Vectorization is exposed to developers, ILS and pipelining are internal

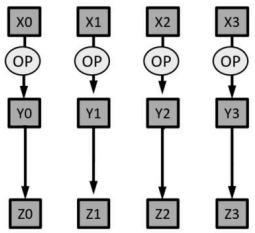
A CPU can vectorize the code below:

```
int[] a = { ... };
int[] b = { ... };
int[] c = new int[16];

for (int i = 0; i < 16; i++) {
    c[i] = a[i] + b[i];
}
```





```
int[] a = { ... };
int[] b = { ... };
int[] c = new int[16];

for (int i = 0; i < 16; i+=4) {
    c[i+0] = a[i+0] + b[i+0];
    c[i+1] = a[i+1] + b[i+1];
    c[i+2] = a[i+2] + b[i+2];
    c[i+3] = a[i+3] + b[i+3];
}
```



Vectorized

```
for (int i = 0; i < 16; i+=4) {
    c[i:i+3] = a[i:i+3] + b[i:i+3];
}
```

Opportunity for higher efficiency: c
instruction once and broadcast it t

→ In this sequential code, the same action is repeated, so the compiler can parallelize

→ The vectorized code leverages the fact we have multiple ALU's → this is the concept of SIMD:

→ SIMD: Single ins., multiple data → same ins. runs on multiple ALUs, the operation is executed in parallel

```
int[] a = { ... };
int[] b = { ... };
int[] c = new int[16];

for (int i = 0; i < 16; i++) {
    c[i] = a[i] + b[i];
}
```

Compiled with –O3

```
movdqa  xmm0, XMMWORD PTR [rsi+rax]
paddd   xmm0, XMMWORD PTR [r8+rax]
movaps  XMMWORD PTR [rdx+rax], xmm0
```

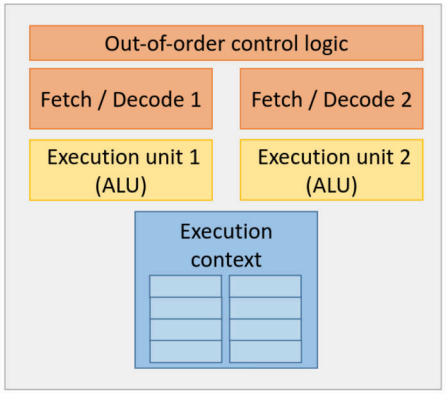
→ movdqa is a SIMD add, moving 4 32-bit ints into SIMD registers

→ paddd is a vector ins. that performs a SIMD add

→ instead of 1 element at a time, we process N elements at a time

Superscalar execution:

→ also ILP, the processor finds independent ins. in an ins. stream and executes them in parallel on different ALU's



Prefetching:

→ Preload ins. into CPU caches before they are needed → max. opportunities for superscalar

→ ins. are independent if their sources and target have different register names and mem. addresses

→ Speculative execution: If we have an independent ins. and we don't know if it will be executed, the CPU can speculatively execute it to keep the functional units busy → Example:

```
x = a + b;
y = c + d;
if (p) 1 = m + n;      // instruction 3: no dependencies
else z = x * y;        // depends on x and y
```

we don't know yet if true or false

Reordering: The compiler (at compile time) and CPU (at runtime) can reorder independent ins.

Example below:

Consider this program executed by two threads:

4 result = f(x);	2 while (done!=true);
1 done = true;	3 print(result);
Core 1:	**Core 2:**
1) dependency analysis	1) dependency analysis
→ independent	→ independent
2) potential instruction reordering	2) potential instruction reordering

synchronize to prevent data race

Consider the following program, executed by two threads:

Note: busy waiting under a lock can cause a deadlock! use wait / notify

```
synchronized {
  result = f(x);
  done = true;
}
```

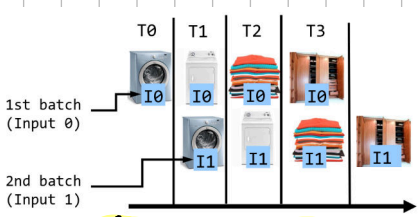
```
synchronized {
   while (done!=true);
   print(result);
}
```

Pipelining: (as I so often say, in the DOCA sheet you will see details) (also, first exam relevant topic)

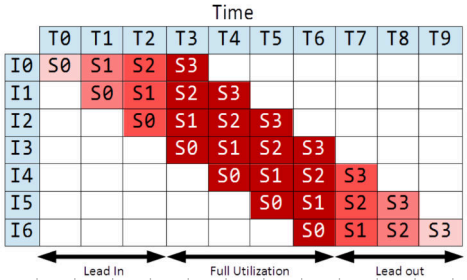
→ Pipelining increases ins. throughput, one ins. can be in each stage at a time

Basic idea:

Pipelining: Washing clothes



Balanced pipeline: all stages take the same time



Def. Throughput: amount of work that can be done by a system in a given period of time

→ In CPUs: # of ins. completed per second, larger is obviously better

→ Throughput bound = $\frac{1}{\max(\text{computationtime}(\text{stages}))}$

Def. Latency: time to perform a computation

→ in CPUs: time required to execute 1 ins. fully in pipeline, lower is obviously better

→ Latency bound: #stages · max(computationtime(stages)) → latency constant over time if pipeline balanced

Washing clothes – Unbalanced pipeline

Takes 5 seconds. We use "w" for Washer next.

Takes 10 seconds. We use "d" for Dryer next.

Takes 5 seconds. We use "f" for Folding next.

Takes 10 seconds. We use "c" for Closet next.

1st attempt:

Time (s) / Load #	0	5	10	15	20	25	30	35	40	45	50	55	60	65	70
Load 1	w	d	d	f	c	c									
Load 2		w	_	d	d	f	c	c							
Load 3			w	_	_	d	d	f	c	c					
Load 4				w	_	_	_	d	d	f	c	c			
Load 5					w	_	_	_	_	d	d	f	c	c	

The total time for all 5 loads is **70 seconds**.

→ we cannot bound the latency, one approach is to make each stage take as long as the longest one → balanced

Now takes 10 seconds.

Takes 10 seconds, as before.

Now takes 10 seconds.

Takes 10 seconds, as before.

Time (s) / Load #	0	10	20	30	40	50	60	70	80	90	100	110	60	65	70	75	80	85	90
Load 1	w	d	f	c															
Load 2		w	d	f	c														
Load 3			w	d	f	c													
Load 4				w	d	f	c												
Load 5					w	d	f	c											

→ bit wasteful, but latency bounded at 40s/load

→ Throughput: 1 load/10s → 6 loads/min → total time for 5 loads: 80s → higher

Step 1: make the pipeline from 1st attempt a bit more fine-grained

Like in the 1st attempt, this takes 5 seconds.

Let's have 2 dryers working in a row. The first dryer is referred to as **d1** and takes 4 seconds, the second as **d2** and takes 6 sec.

Like in the 1st attempt, it takes 5 seconds.

Let's have 2 closets working in a row. The first closet is referred to as **c1** and takes 4 seconds, the second as **c2** and takes 6 sec.

Note: splitting slow stages into two pipeline stages, not parallel dryers / cupboards

Now, balance the new pipeline

Step 2: and also, like in the 2nd pipeline, make each stage take as much time as the longest stage does from Step 1 [this is 6 seconds due to d2 and c2]

It now takes 6 seconds.

Each of d1 and d2 dryers take 6 seconds.

Now takes 6 seconds.

Each of c1 and c2 closets now take 6 seconds.

Note: splitting slow stages into two pipeline stages, not parallel dryers / cupboards

3rd attempt:

Time (s) / Load #	0	6	12	18	24	30	36	42	48	54	60	110	60	65	70	75	80	85	90
Load 1	w	d1	d2	f	c1	c2													
Load 2		w	d1	d2	f	c1	c2												
Load 3			w	d1	d2	f	c1	c2											
Load 4				w	d1	d2	f	c1	c2										
Load 5					w	d1	d2	f	c1	c2									

→ latency bound: 6*6 = 36s

→ Throughput: 1 load/6s → 10 loads/min → total time: 60s

→ Throughput optimization can increase latency: for example splitting the dryers could have made them take longer

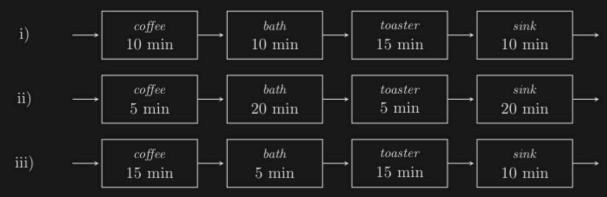
→ Pipelining adds constant time overhead between stages (communication)

→ Infinitely small stages not practical / execution time of a task may take longer with pipelining than serially

Pipeline EXAMple: (F521)

2. Drei Studenten leben in einer Wohngemeinschaft. Seit dem neuen Semester hat sich der Stundenplan geändert und nun müssen alle gleichzeitig an der Uni sein. Leider funktioniert die bisherige Regelung nicht mehr, dass jeder die gemeinsamen Räume nutzen kann wie er möchte. Alle sind sich einig, dass der Tag mit einem Kaffee ("coffee") starten muss, direkt gefolgt von der Dusche ("bath"). Danach setzt der Hunger ("toaster") ein und zu guter letzt sollte natürlich alles ordentlich hinterlassen werden und das Geschirr gespült sein ("sink"). Betrachten Sie die drei unten dargestellten separaten Pipelines. **Nehmen Sie für alle nachfolgenden Fragen an, dass die drei Studenten kein Objekt gleichzeitig benutzen wollen.** Die folgenden drei Zeilen zeigen drei mögliche separate Pipelines:

Three students live in a shared apartment. With the start of the new semester the class schedule changed, so they now all have to be at the university at the same time. Unfortunately, the existing rule allowing everyone to use the shared rooms as they want does not work anymore. They all agree that the day starts with a coffee ("coffee"), followed by a shower ("bath"). Then hunger ("toaster") kicks in and at the end everything should be clean again and the dishes be done ("sink"). Consider the three separate pipelines shown below. ***For all of the following question, assume that the three students do not want to use an object at the same time.*** *The following three lines show three possible separate pipelines:*



(c) Für Pipeline ii): Was ist der Speedup verglichen mit der streng sequentiellen Reihenfolge (d.h. wenn jeder Student erst anfängt, wenn der Vorherige mit dem Geschirrspülen fertig ist, statt schon anzufangen wenn die Kaffeemaschine frei ist)? Begründen Sie Ihre Antwort und geben Sie alle Rechenschritte an. Sie können das Ergebnis als Bruch angeben. (3)

For pipeline ii): What is the speedup compared to the strictly sequential order (i.e., every student only starts after the previous student finished doing the dishes, instead of starting as soon as the coffee maker is available)? Justify your answer and state all the calculation steps. You can provide the result as a fraction.

(a) Was ist der Durchsatz ("throughput") der einzelnen Pipelines pro Stunde in dieser WG? (3)

What is the throughput of each pipeline per hour in this living community?

i) 4 ii) 3 iii) 3

→ remember the formula and the unit of time (hour)

(b) Das Ziel ist, dass alle drei Studenten so schnell wie möglich fertig sind. Rechnen sie die Zeit aus, die es benötigt bis alle drei Studenten die Pipelines durchlaufen haben. Begründen Sie Ihre Antwort und geben Sie alle Rechenschritte an. Welche Pipeline(s) ist (sind) die schnellste(n)? (5)

The goal is for all students to finish the morning schedule as quickly as possible. Calculate the time it takes to finish each Pipeline for all the three students. Justify your answer and state all the calculation steps. Which pipeline(s) is (are) the fastest?

Cool formula: Total time = all stages except longest stage * (longest stage * students)

→ i): 15 + (10+10+10) = 75 min, analogously, ii): 90, iii): 75

→ i) and iii) are the fastest

→ purely sequential takes (5+20+5+20)*3 = 150 min

→ Speedup formula: $\frac{t_{sequential}}{t_{pipelined}} = \frac{150}{90} = \frac{5}{3}$

You may have to think ahead a little:

(d) Aufgrund von einer Pandemie ist es den Studenten leider nicht erlaubt eines der 4 gemeinsamen Objekte zu benutzen bevor der vorhergehende Student komplett fertig ist (d.h. mit dem "sink" Schritt abgeschlossen hat). Die Studenten haben jedoch die Möglichkeit, jeden Schritt um 10 Minuten zu verlängern um die Objekte zu desinfizieren, damit die nächste Person das Objekt unmittelbar nach der Desinfektion verwenden kann. Welche Pipeline ist am schnellsten mit dieser neuen Regelung und lohnt es sich für die Studenten diese zu implementieren? Aus Solidarität muss der letzte Student ebenso alles desinfizieren. Begründen Sie Ihre Antwort und geben Sie alle Rechenschritte an. (4)

Due to a pandemic the students are not allowed to use one of the 4 shared objects before the previous student is finished with all of them (i.e. finished the "sink" step). However, students have the option to extend each step by 10 minutes for disinfection of the objects to allow the next person to use the object immediately after disinfection. Which pipeline is fastest with this new rule? Is it worth it for the students to implement the rule? In solidarity, the last student must also disinfect everything. Justify your answer and state all the calculation steps.

→ now imagine that we add 10 min to each step in each pipeline:

→ time for i): 25*3 + 60 = 135 min, ii): 30*3 + 60 = 150 min, iii): 25*3 + 60 = 135 min

→ this happens to equal each of the pipelines' sequential times without the rule ⇒ it does not matter if we implement the rule (see, free points, right?)

Amdahl & Gustafson:

→ Motivation: when is it worth it to throw more cores at a computational problem?

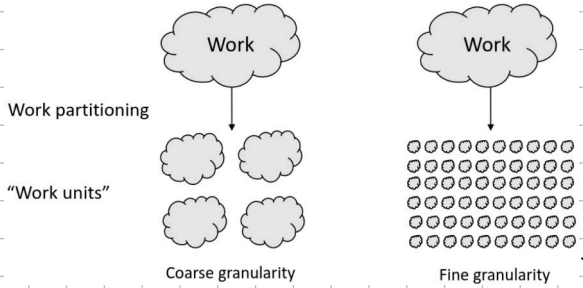
→ more cores doesn't always cause a linear speedup, fast != efficient, think of cost/energy tradeoffs

Scalability:

→ some parts of the program will depend on each other → this sequential part dominates at high core counts and limits speedup

→ this also depends on data structures and algorithms → data structures like linked lists or trees require seq. access

→ algorithms can be restructured to allow for parallelism



→ workloads may be imbalanced → some cores are idle while others overloaded

→ goal: full utilization → programmer has to map workloads to threads

→ how quick is assignment or reassignment of tasks to threads? is it worth the overhead?

→ Overhead: time spent on synchronization, locking, context switching

→ high contention or excessive synchronization can bottleneck parallel efficiency

→ Memory access contention: if all cores access

shared memory, memory bandwidth becomes bottleneck → for badly partitioned data, more cores ⇒ more cache misses

Parallel Performance (the exam-relevant math!)

→ we get **speedup** when adding processors, what happens if processors → ∞? If a program scales linearly, we get linear speedup

→ Let's call the seq. execution time T_1 and the execution time for p CPUs T_p

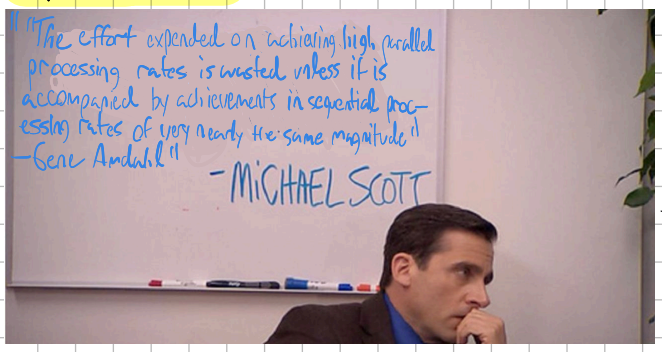
→ if $T_p = T_1/p$, that is perfect (linear), $T_p > T_1/p$ is performance loss and the normal case, $T_p < T_1/p$ is impossible

→ (parallel) speedup S_p on p CPUs: $S_p = T_1/T_p$ → $S_p = p$ is linear speedup, $S_p < p$ is normal sub-linear speedup, $S_p > p$ sorcery

→ Efficiency: S_p/p

→ when actually writing parallel code, use your optimized serial program as a baseline for T_1

Amdahl's Law:



→ Execution time T_1 of a program falls into **parallelizable work** (W_{par}) and **non-parallelizable work** (W_{ser})

$\Rightarrow T_1 = W_{ser} + W_{par} \Rightarrow T_p \geq W_{ser} + \frac{W_{par}}{p}$

→ if you use this to bound speedup, you get **Amdahl's Law:**

$$S_p \leq \frac{W_{ser} + W_{par}}{W_{ser} + \frac{W_{par}}{p}}$$

Corollary: Let f be the non-parallelizable fraction of the total work: $W_{ser} = fT_1$, $W_{par} = (1-f)T_1$

$$\Rightarrow S_p \leq \frac{1}{f + \frac{1-f}{p}}$$

→ Wording like "minimize runtime" or "constant work" (or fixed) implies Amdahl on the exam

→ **Scaling with clock speed:** if you are given clock speeds on the exam, Speedup' = Clock speed in GHz * Speedup

→ this **EXAMple** made me aware of this (also FS21):

(d) Nehmen Sie an, dass Ihr Budget Ihnen ermöglich 5 Prozessoren mit einer Taktung von 2.5 GHz oder 8 Prozessoren mit einer Taktung von 1.5 GHz zu kaufen. Angenommen die Arbeit ist konstant und der parallelisierbare Anteil entspricht 50%, welche der beiden Optionen ist die bessere? (4)

- A: 8 Prozessoren mit einer Taktung von 1.5 GHz.
- B: 5 Prozessoren mit einer Taktung von 2.5 GHz.

Assume that your budget allows to buy 5 processors with a clock speed of 2.5 GHz or 8 processors with a clock speed of 1.5 GHz. Assuming that the work (problem size) is constant and the parallelizable part is equal to 50%, which of the two options is better?

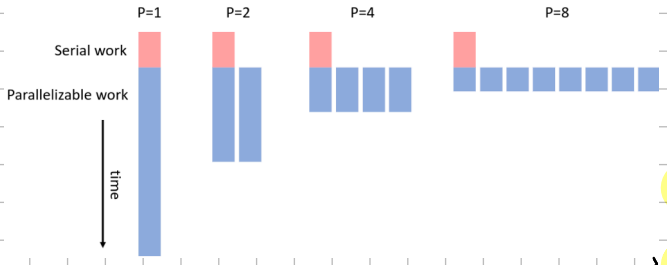
- *A: 8 processors with a clock speed of 1.5 GHz.*
- *B: 5 processors with a clock speed of 2.5 GHz.*

→ $\text{Speedup}_A \leq \frac{1}{\frac{1}{2} + \frac{1}{8}} = \frac{1}{\frac{9}{16}} = \frac{16}{9}$

→ $\text{Speedup}_B \leq \frac{1}{\frac{1}{2} + \frac{1}{5}} = \frac{1}{\frac{7}{10}} = \frac{5}{3}$

→ $\text{Speedup}'_A = \frac{3}{2} \cdot \frac{16}{9} = \frac{8}{3}$, $\text{Speedup}'_B = \frac{5}{2} \cdot \frac{5}{3} = \frac{25}{6} > \frac{8}{3} \Rightarrow$ B better

→ with infinite cores, $S_\infty \leq \frac{1}{f}$ → illustrated:



→ Amdahl's view was **pessimistic**, scalability is limited

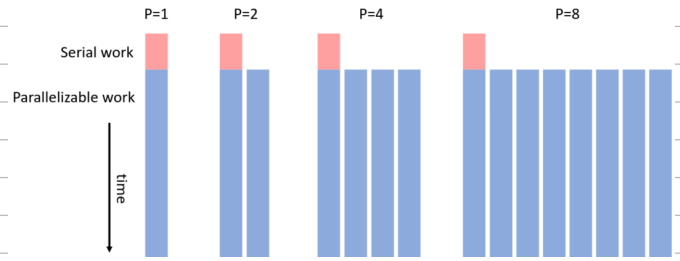
→ all non-parallel parts of a program limit speedup → emphasizes parallel algorithms

Gustafson's Law:

→ **Optimistic view**, some considerations (for the exam) when we can use it:

→ when runtime, not problem size is constant, more processors allow us to solve larger problems in the same time

⇒ parallel part of a program scales with problem size → Illustrated:



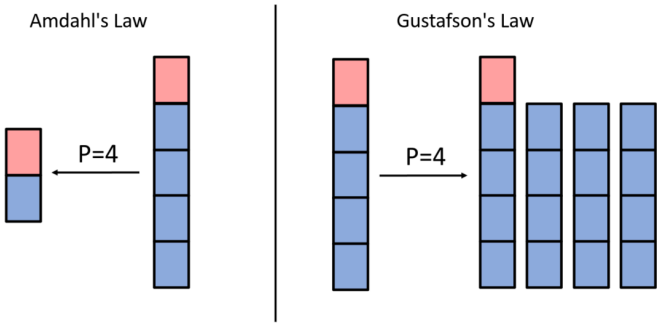
→ let f be the seq. part, formula:

$W = P(1-f)T_{wall} + fT_{wall}$

$\Rightarrow S_p = f + P(1-f) = P - f(P-1)$ (for fixed time)

→ on the exam, "best use of runtime" implies Gustafson

Amdahl's vs Gustafson's Law



Divide and Conquer:

→ Java Threads are inefficient because:

→ the programmer has to manually partition work, which is tedious, can lead to load imbalance, and scales badly

→ manually managing thread assignment (start/join) rarely optimal thread assignment (order)

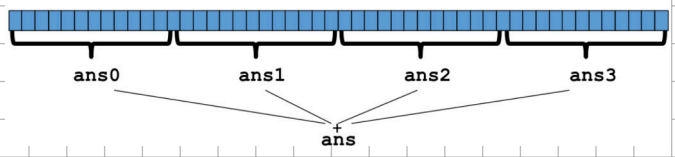
→ because we want the programmer to focus on designing parallel algos., thread assignment and user-level scheduling is handled by frameworks

Code example: summing over an array:

→ Sequential version (baseline):

```
public static int sum(int[] input){
    int sum = 0;
    for(int i = 0; i < input.length; i++){
        sum += input[i];
    }
    return sum;
}
```

Naive parallel idea: have 4 threads simultaneously sum 1/4 of the array



→ Create 4 threads, give them 1/4 of work, call start on all, join to wait on results, sum those

Code:

```
class SumThread extends java.lang.Thread {

    int lo; // arguments
    int hi;
    int[] arr;

    int ans = 0; // result

    SumThread(int[] a, int l, int h) {
        lo=l; hi=h; arr=a;
    }

    public void run() { //override must have this type
        for(int i=lo; i < hi; i++)
            ans += arr[i];
        }
}
```

use fields (variables) to communicate between threads (i.e. are overriding run() without result or arguments)

```
class SumThread extends java.lang.Thread {
    int lo, int hi, int[] arr; // arguments
    int ans = 0; // result
    SumThread(int[] a, int l, int h) { … }
    public void run(){ … } // override
}

int sum(int[] arr){
    int len = arr.length;
    int ans = 0;
    SumThread[] ts = new SumThread[4];
    for(int i=0; i < 4; i++){ // do parallel computations
        ts[i] = new SumThread(arr,i*len/4,(i+1)*len/4);
        ts[i].start(); // start the threads
    }
    for(int i=0; i < 4; i++) { // combine results
        ts[i].join(); // wait for helper to finish! -> try catch
        ans += ts[i].ans;
    }
    return ans;
}
```

→ This style is called fork-join programming

→ reduces race conditions by encouraging independent tasks (keep in mind we are using shared memory)

Why is this a bad parallel algorithm?

→ we want reusable code that is efficient across platforms (forward-portable with growing core count) → we can parametrize by #threads:

```
int sum(int[] arr, int numTs){
    int ans = 0;
    SumThread[] ts = new SumThread[numTs];
    for(int i=0; i < numTs; i++){
        ts[i] = new SumThread(arr,(i*arr.length)/numTs,
                              ((i+1)*arr.length)/numTs);
        ts[i].start();
    }
    for(int i=0; i < numTs; i++) {
        ts[i].join();
        ans += ts[i].ans;
    }
    return ans;
}
```

→ this is still poor because we need to commit to a number of threads before running

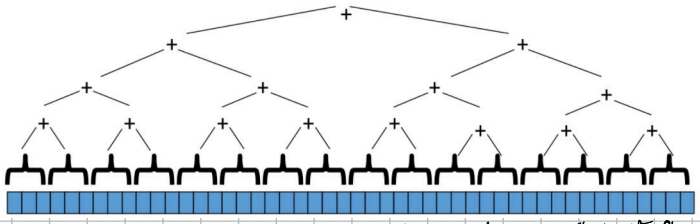
→ numThreads == numProcessors not good if processors needed for other tasks

→ needs to be made dynamic

→ also, we don't consider that workloads can vastly differ in time taken (not for sum)

→ best practice: expose as much parallelism as possible, let JVM runtime efficiently distribute work across threads

→ we can still do this with standard threads and Divide & Conquer → parallel recursive calls



yes i'm naturally using an em-dash, maybe I'm ChatGPT after all?

recall the pattern from A&D:

```
Divide and Conquer:
  if cannot divide:
    return unitary solution (stop recursion)
  divide problem in two:
  solve first (recursively)
  solve second (recursively)
  combine solutions
  return result
```

Sequential recursive sum — code:

```
public static int do_sum_rec(int[] xs, int l, int h) {
    int size = h-l;
    if (size == 1)  /*check for termination criteria*/
        return xs[l];

    /* split array in half and call self recursively*/
    int mid  = size / 2;
    int sum1 = do_sum_rec(xs, l, l + mid);
    int sum2 = do_sum_rec(xs, l + mid, h);
    return sum1 + sum2;
}
```

Parallel Recursive Sum:

```
public class SumThread extends Thread {
int[] xs;
int h, l;
int result;

public SumThread(int[] xs, int l, int h){
    super();
    this.xs = xs;
    this.h = h;
    this.l = l;
}

public void run(){
    /* Do computation and write to result */
    return;
}
```

```
public void run(){
    int size = h - l;
    if (size == 1) {
        result = xs[l];
        return;
    }
    int mid = size / 2;
    SumThread t1 = new SumThread(xs, l, l + mid);
    SumThread t2 = new SumThread(xs, l + mid, h);

    t1.start();
    t1.join();

    t2.start();
    t2.join();

    result = t1.result + t2.result;
    return;
}
```

Is this OK? -> no parallelism!

```
public void run(){
    int size = h - l;
    if (size == 1) {
        result = xs[l];
        return;
    }
    int mid = size / 2;
    SumThread t1 = new SumThread(xs, l, l + mid);
    SumThread t2 = new SumThread(xs, l + mid, h);

    t1.start();
    t2.start();

    t1.join();
    t2.join();

    result = t1.result + t2.result;
    return;
}
```

Remark: This doesn't compile because join() can throw exceptions. We need a try-catch block here.

Result: OutOfMemoryError: unable to create native thread:

→ Reason: Java threads are mapped 1 to 1 to OS threads → quite heavyweight

→ creating a thread per small task is very inefficient

→ Manual fixes (suboptimal): (going down to single element computation creates massive overhead)

→ use a sequential cutoff (usually 500–1000) → eliminates most recursive thread creation (bottom levels of tree)

→ do not create 2 recursive threads, create one and do work yourself → involves created threads again

Working Java Thread D&C solution:

```
public void run(){
    int size = h-l;
    if (size < SEQ_CUTOFF)
        for (int i = l; i < h; i++)
            result += xs[i];
    else {
        int mid = size / 2;
        SumThread t1 = new SumThread(xs, l, l + mid);
        SumThread t2 = new SumThread(xs, l + mid, h);
        t1.start();
        t2.start();
        t1.join();
        t2.join();
        result = t1.result + t2.result;
    }
}
```

cutoff, eliminating bottom levels of tree

```
int mid = size / 2;
SumThread t1 = new SumThread(xs, l, l + mid);
SumThread t2 = new SumThread(xs, l + mid, h);

t1.start();
t2.start();

t1.join();
t2.join();

result = t1.result + t2.result;
```

start two threads in each recursive step -> wasteful

```
int mid = size / 2;
SumThread t1 = new SumThread(xs, l, l + mid);
SumThread t2 = new SumThread(xs, l + mid, h);

t1.start();
t2.run();
t1.join();

result = t1.result + t2.result;
```

only start one thread, call run() directly on t2 object -> better performance, but we once said that we should never call run() directly...

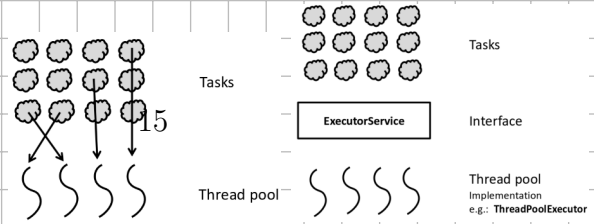
→ built-in support for FJ parallelism would make us not have to manually optimize like this

→ we did solve fine because we can combine results in O(log n) instead of O(n) with enough processors

→ D&C takes advantage of associative operations

Executor Service:

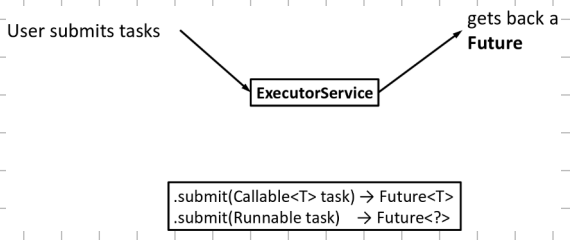
→ Alternatively, we can schedule tasks onto threads



→ Java's ExecutorService manages asynchronous tasks

15

General idea:



→ ExecutorService can handle Runnable or Callable tasks (wait, scala-stm does this the same way, keep that in mind!)

Interface Runnable:
→ void run() ⟶ Does not return result

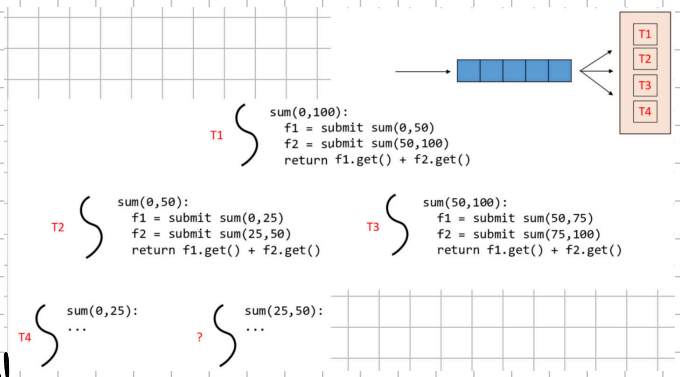
Interface Callable<T>:
→ T call() ⟶ Returns result

Recursive sum using ExecutorService:

```
public Integer call() throws Exception {
    int size = h - l;
    if (size == 1)
        return xs[l];

    int mid = size / 2;
    sumRecCall c1 = new sumRecCall(ex, xs, l, l + mid);
    sumRecCall c2 = new sumRecCall(ex, xs, l + mid, h);

    Future<Integer> f1 = ex.submit(c1);
    Future<Integer> f2 = ex.submit(c2);

    return f1.get() + f2.get();
}
```

→ looks simple, huh? what if I told you it never returns?

→ but why? as you see below, we run out of threads and the tasks wait forever



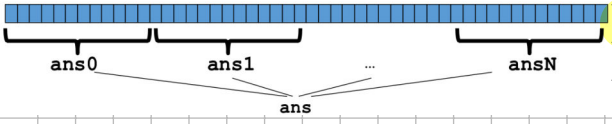
→ ExecutorService is inadequate for recursive problems where deep structures require waiting on partial results

→ It's well-suited for flat structures or tasks that run independently in parallel

→ If we really wanted to solve this with ExecutorService, we need to change our approach from D&C.

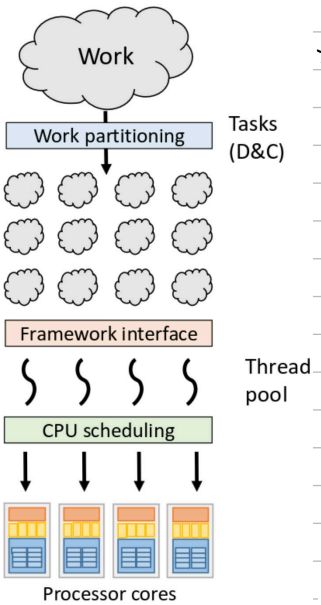
→ possible approach: decouple work partitioning from solving the problem

→ split array into chunks, create task per chunk, submit tasks into ExecutorService, combine results (sum) → hard to sum and partition in par.



ForkJoin framework:

→ designed to meet needs of D&C fork-join parallelism

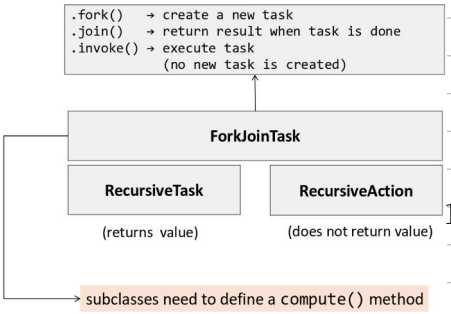


→ Idea: Partition work into tasks using D&C, submit FJ task to FJ interface, framework assigns tasks to FJ thread pool

Using the framework is similar to threads, you just need a little setup like creating a ForkJoinPool:

→ instead of making a subclass of Thread, make a subclass of RecursiveTask<V>
overriding run(), override compute()
using an ans field, return a V from compute()
calling start(), call fork()
calling join(), call join() that returns an answer
calling run() to hand-optimize, call compute() to hand-optimize
having a topmost call to run(), create a pool and call invoke()

Tasks in FJ:



Usual procedure:

```
t.fork()      ⟶ spawn a new task
...           ⟶ possibly do other things
res = t.join() ⟶ wait for task result
```

```
t.invoke()    ⟶ execute the task computation without spawning a task (in-place)
```


16

Recursive Sum using FJ:

```
class Globals {
    static ForkJoinPool fjPool = new ForkJoinPool();
}

static long sumArray(int[] array) {
    return Globals.fjPool.invoke(new SumForkJoin(array,0,array.length));
}
```

Default # of processors

ForkJoinPool:

→ Constructor create number of threads = number of available processors

→ invoke() submits task and waits until completion, submit() submits task and receives a future

```
class SumForkJoin extends RecursiveTask<Long> {
    int low;
    int high;
    int[] array;

    SumForkJoin(int[] arr, int lo, int hi) {
        array = arr;
        low   = lo;
        high  = hi;
    }

    protected Long compute() { /*…*/ }
```

```
protected Long compute() {
    if(high - low <= 1)
        return array[high];
    else {
        int mid = low + (high - low) / 2;
        SumForkJoin left  = new SumForkJoin(array, low, mid);
        SumForkJoin right = new SumForkJoin(array, mid, high);
        left.fork();
        right.fork();
        return left.join() + right.join();
    }
}
```

```
protected Long compute() {
    if(high - low <= SEQUENTIAL_THRESHOLD) {
        long sum = 0;
        for(int i=low; i < high; ++i)
            sum += array[i];
        return sum;
    } else {
        int mid = low + (high - low) / 2;
        SumForkJoin left  = new SumForkJoin(array, low, mid);
        SumForkJoin right = new SumForkJoin(array, mid, high);
        left.fork();
        long rightAns = right.compute();
        long leftAns  = left.join();
        return leftAns + rightAns;
    }
}
```

We are splitting array to size=1, overhead

Make sure each task has 'enough' to do!

Framework tries to perform well even if the tasks do not have so much computation

Pushes task to the local queue, making it stealable

Optimize, call compute: Executes task immediately in the current thread (avoids queuing)

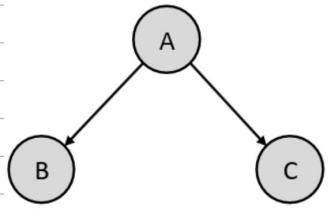
→ Parallelism only helps if each task does enough work to justify the overhead of creating and scheduling tasks

→ tasks should perform ~100–10000 basic operations before being split → sequential threshold

Task graphs:

→ Tasks execute code, spawn other tasks (start in Java Thread, submit in ExecService, or fork in FJ), wait for results from other tasks (join in JT and FJ, get in ExS)

→ Spawning tasks creates a graph:



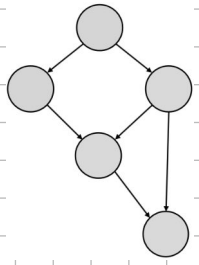
→ edges show: B and C created by A

Program executions using fork/join can be seen as a directed acyclic graph (DAG):

→ Nodes: units of work → tasks in FJ

→ Edges: dependencies

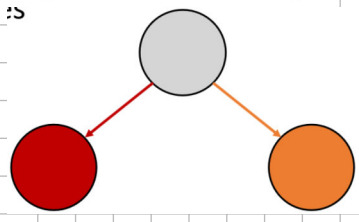
→ DAG shows who must finish before who



→ fork ends a node, creates 2 outgoing edges:

→ one to a new forked task, another to a continuation of the current task

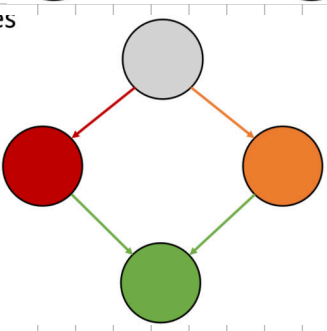
→ can execute in parallel if worker thread available



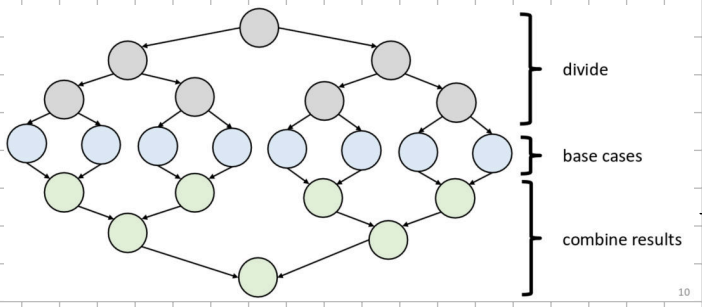
→ join creates a node with two incoming edges:

→ one comes from finished forked task, other from continuation of original task which was waiting

→ continuation only executes when both branches leading into join() are complete



Balanced tree D&C example: (used in many algorithms)



→ Task graph is dynamic (develops at runtime)

→ Tasks can execute in parallel but don't have to → tasks are dynamically assigned to available worker threads

→ work-stealing allows for automatic workload balancing because idler threads can steal tasks from busier tasks

Simple example: Parallel Fibonacci:

fib() function

$$fib(n) = \begin{cases} n, & \text{if } n \leq 1 \\ fib(n-1) + fib(n-2), & \text{if } n > 1 \end{cases}$$

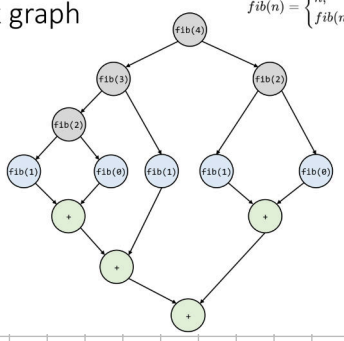
Sequential version

```
public class SequentialFib {
    public static int fib(int n) {
        if (n <= 1) return n;

        return fib(n - 1) + fib(n - 2);
    }
}
```

Parallel version (FJ)

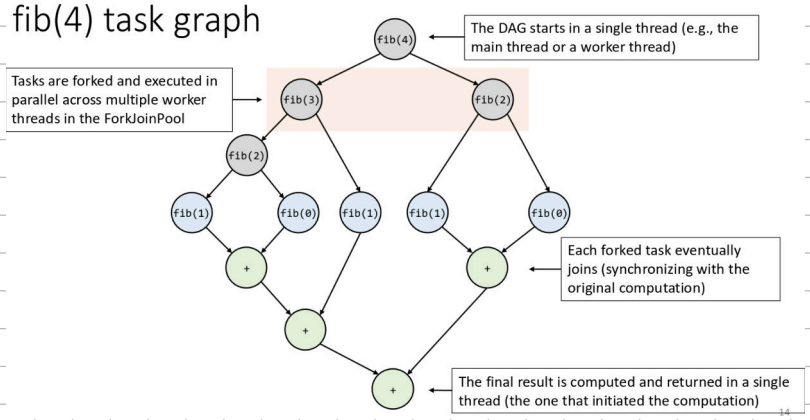
```
public class ParallelFib extends RecursiveTask<Integer> {
…
protected Integer compute {
    if (n <= 1) return n;
    ParallelFib f1 = new ParallelFib(n - 1);
    ParallelFib f2 = new ParallelFib(n - 2);

    f1.fork();              // Fork the first subproblem
    int y = f2.compute(); // Compute the second subproblem in
                             the current thread
    int x = f1.join();      // Join the first subproblem's result
    return x + y;
}
```

fib(4) task graph




```
if (n <= 1) return n;

…f1 = new …(n - 1);
…f2 = new …(n - 2);

f1.fork();
int y = f2.compute();
int x = f1.join();

return x + y;
```

fib(4) task graph



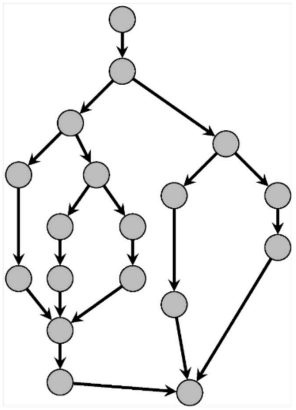
→ DAG captures task dependencies and potential parallelism, but says nothing about exec. time

→ Recall: T_1 (work) → total sequential exec. time, tells us total work (sum of all task exec. times)

T_p (parallel time) → exec. time on P threads, shows real parallel performance and speedup

→ T_∞ (span): exec. time assuming infinite threads, shows best possible speedup → longest dependency chain that cannot be parallelized

In the graph:



→ T_1 is the sum of the time cost of all nodes in graph, as if we executed it sequentially

→ T_p: we can control p by configuring the thread pool (ForkJoinPool), but we cannot control T_p

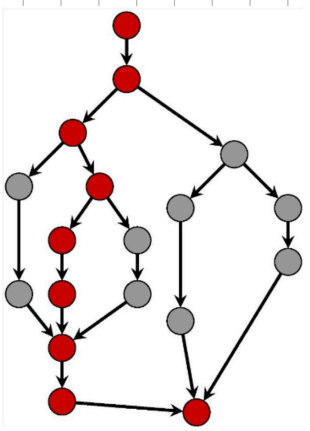
→ T_p depends on task dependencies in DAG, work distribution and load balancing, thread contention and OS-level scheduling

→ T_∞ is the critical path length → longest chain of dependent ops. in DAG

→ Parallelism = T_1 / T_∞ → upper bound to speedup

→ here, $T_1 / T_\infty = 2$ → the span is the bottleneck to any more parallelism

On this graph, T_1 is 18



On this graph, T_∞ is 9

→ Intuitively: wide task graph → higher potential para, deep task graph → more seq. dependencies (longer T_∞, less speedup)

→ speedup limited by ratio T_1 / T_∞ (work vs. critical path → Amdahl) → parallelism limited by dependencies

Scheduling task graphs:

→ Scheduler is an algorithm assigning tasks to worker-threads → T_p depends on it, T_1, T_∞, p depend on programmer